CASE STUDY DECK

사례로 살펴보는

Agile 방법론과 MSA 개발 이해

실전 사례로 이해하는 아키텍처 선택과 팀 협업 방식

SKALA 임성열 교수 Ph.D.

이 자료를 보는 이유

:: CASE STUDY DECK · 4 PARTS

--overview

실습을 하다 보면 "왜 이렇게까지 나눠서 개발하지?", "MSA가 항상 정답인가?" 같은 의문이 생깁니다. 이 자료는 실제 기업 사례와 우리 실습 프로젝트를 통해 그 답을 함께 찾아갑니다. 사례마다 "더 깊이 보기"와 "우리 프로젝트 적용" 슬라이드를 함께 두어, 읽는 데서 그치지 않고 바로 팀 과제에 대입해볼 수 있게 했습니다.

예습 · 복습용으로 이렇게 활용하세요

수업 전 : 각 부 "메인 사례" 슬라이드만 먼저 읽기
수업 후 : "더 깊이 보기·적용" 슬라이드로 우리 프로젝트에 대입
막힐 때 : 4부 체크리스트로 스스로 진단하기

① **아키텍처는 정답이 아니다**
산업 사례로 보는 선택의 기준 · 15장

② **우리 프로젝트의 Agile×MSA**
실습에서 실제로 드러나는 원리 · 3장

③ **흔한 실수와 극복**
이번 학기 실제 사례 · 3장

④ **스스로 점검하기**
자가진단 체크리스트 · 1장

완벽한 설계 후 개발했다가 무너진 프로젝트

PART 1 :: CASE 01/05

① 아키텍처 선택 | ② 우리 프로젝트 | ③ 실수와 극복 | ④ 자가진단

--case · waterfall

1

요구사항 확정

개발 전 모든 기획을 완벽히 확정

→

2

순서대로 개발

설계 → 구현 → 테스트를 한 방향으로 진행

→

3

도중 요구사항 변경

시장·고객 반응에 따라 계획이 바뀜

→

4

전면 재작업

이미 만든 부분까지 다시 뜯어고쳐야 함

Agile은 이 문제를 "한 번에 완벽하게"가 아니라 "작게 나눠서 자주, 피드백을 반영하며"로 풀어낸 방법론입니다.

정부 프로젝트도 예외는 아니었다

PART 1 :: CASE 01 · DEEP DIVE

① 아키텍처 선택 | ② 우리 프로젝트 | ③ 실수와 극복 | ④ 자가진단

--deep-dive

2001년 9·11 테러 이후 FBI는 노후화된 수사 기록 시스템을 교체하는 "버추얼 케이스 파일(Virtual Case File)" 프로젝트를 시작했습니다. 방대한 요구사항 명세서를 먼저 확정하고 그대로 개발을 진행했지만, 프로젝트가 진행되는 수년 동안 수사 현장의 요구사항은 계속 바뀌었고 기술 환경도 변했습니다. 완성 시점에는 이미 설계 자체가 시대에 뒤처져 있었고, 결국 시스템은 실전 배치 없이 폐기되었습니다.

요구사항 문서가 두꺼울수록 안전하다고 믿었지만, 그 문서를 다 지킨 결과는 폐기였습니다.

$1.7억+ 투입 비용

4년+ 개발 기간

0건 실전 배치

Executive Summary : 비즈니스 가치와 기술과 연결점

PART 1 :: CASE 01 · DEEP DIVE

1. FBI의 버추얼 케이스 파일(VCF) 사례 연구

시장과 기술의 변화를 예측하지 못한 채 고정된 설계에 집착하는 '워터폴(Waterfall) 방식'의 치명적 한계를 시사합니다. 변화무쌍한 비즈니스 가치에 즉각 발맞추기 위해서는 인프라와 소프트웨어 구조 역시 유연하게 쪼개고 진화시킬 수 있는 마이크로서비스 아키텍처(MSA)적 접근이 필수적입니다.

2. VCF 실패가 주는 비즈니스 및 기술적 교훈

- **비즈니스 가치의 동적 변화**
  - 수사 현장(시장)의 요구사항은 살아있는 생물처럼 실시간으로 변화함
  - 한 번 정한 기획서에 기술을 끼워 맞추면 완성 시점엔 이미 무가치한 유물이 됨
- **모놀리식(Monolithic) 탑 쌓기의 위험성**
  - 거대한 시스템을 통째로 기획하고 한 번에 오픈하려는 빅방(Big Bang) 방식의 한계
  - 하나의 설계 오류나 요구사항 변경이 전체 아키텍처를 뒤흔드는 부작용 발생
- **기술 환경의 유통기한 고려**
  - 수년간 개발하는 동안 하드웨어와 소프트웨어 생태계는 다음 세대로 진화함
  - 기술 부채를 유연하게 털어내지 못하는 경직된 구조는 시스템 전체의 폐기로 이어짐

우리 프로젝트에서는 이렇게 피한다

PART 1 :: CASE 01 · APPLY TO OUR PROJECT

--apply

우리 실습은 스프린트 구조 자체가 이 실패를 피하도록 설계되어 있습니다. 팀 스스로 3가지를 점검해보세요.

1. 스프린트1을 짤 때, "이 기능 없이 시연이 가능한가?"만 남기고 나머지는 스프린트2로 미뤘는가?
2. 스프린트1 시연 후 팀 밖의 피드백(다른 팀•강사)을 실제로 받았는가?
3. 그 피드백을 스프린트2 계획에 반영해 우선순위를 다시 정렬했는가?

체크리스트 3개에 모두 "예"라고 답할 수 있다면, 이미 VCF와 다르게 안전한 방향으로 가고 있는 것입니다.

Netflix, 3일간의 장애가 만든 전환점

PART 1 :: CASE 02/05

① 아키텍처 선택 | ② 우리 프로젝트 | ③ 실수와 극복 | ④ 자가진단

--case · monolith to msa

2008년, 데이터베이스 손상으로 서비스 전체가 3일간 다운되는 사고가 발생했습니다. 하나의 장애가 전체를 마비시키는 모놀리식 구조의 한계를 절감한 Netflix는 이후 수년에 걸쳐 클라우드 기반 마이크로서비스로 하나씩 서비스를 옮겨갔고, 2016년 마지막 결제 시스템까지 전환을 완료했습니다.

가용성 · 확장성 · 배포 속도를 얻기 위한 선택이었습니다.

700+ 마이크로서비스

150억+ 일일 API 호출

190개+ 서비스 제공 국가

장애를 두려워하지 않는 문화까지

PART 1 :: CASE 02 · DEEP DIVE

① 아키텍처 선택 | ② 우리 프로젝트 | ③ 실수와 극복 | ④ 자가진단

--deep-dive

2008

장애 발생

DB 손상으로 3일간 전체 다운, 클라우드 이전 결정

→

2009~11

단계적 재작성

서비스를 하나씩 마이크로서비스로 이전

→

2011

Chaos Monkey

운영 중 서버를 무작위로 종료시켜 장애 대응력 상시 검증

→

2016

전환 완료

마지막 결제 시스템까지 마이크로서비스 이전 마무리

Netflix는 서비스를 나누는 데서 그치지 않고, "장애가 나도 버티는지"를 상시 검증하는 문화까지 만들었습니다.

Executive Summary : 비즈니스 전략과 MSA 기술의 만남

PART 1 :: CASE 02 · DEEP DIVE

1. 비즈니스 모델의 근본적인 변화 (DVD ➔ 스트리밍)

넷플릭스는 2007년에 처음으로 인터넷 스트리밍 서비스를 도입했습니다.

•DVD 배송 시절: 고객과의 상호작용이 일주일에 한두 번(주문 및 반납)에 불과했습니다. 기존 온프레미스(자체 데이터센터) 인프라로도 관리가 가능했습니다.

•스트리밍 시대: 고객이 매일, 실시간으로 사이트에 접속해 영상을 재생하므로 데이터센터 트래픽이 고객당 무려 1,000배 이상 폭증하기 시작했습니다.

2. 자체 데이터센터의 물리적 한계

스트리밍 서비스가 폭발적으로 성장하면서, 기존 데이터센터 방식으로는 밀려드는 사용량을 감당할 수 없었습니다.

•인프라를 늘리려면 서버 장비를 새로 주문하고, 조립하고, 데이터센터에 설치하는 데만 수개월이 걸렸습니다.

•급증하는 사용자의 접속 속도를 맞추기 위해 실시간으로 서버를 늘리고 줄이는 '탄력성(Elasticity)'이 절실했습니다.

3. "우리가 잘하는 것에 집중하자"는 전략

넷플릭스 경영진은 자신들의 핵심 역량이 '데이터센터를 짓고 서버를 관리하는 기술'이 아니라 '고객에게 최고의 콘텐츠와 시청 경험을 제공하는 것'이어야 한다고 판단했습니다.

따라서 하드웨어 인프라 관리는 아마존 웹 서비스(AWS)라는 클라우드 전문 기업에 전적으로 맡기고, 자신들은 서비스 고도화와 영상 인코딩, 추천 알고리즘 개발에만 집중하기로 한 것입니다.

비즈니스 가치와 MSA 기술의 결합이 필요한 이유

넷플릭스가 글로벌 스트리밍 기업으로 체질을 바꿀 수 있었던 것은 **비즈니스 전략에 맞춰 기술 구조를 잘게 쪼개어 변화에 대응(MSA)했기 때문**입니다. **반면 FBI는 거대하고 단단한 성벽을 쌓으려다 무너졌**습니다.

급변하는 현대 비즈니스에서 살아남으려면 "요구사항은 무조건 바뀐다"라는 전제하에, **비즈니스 기능 단위로 서비스를 격리하고 독립적으로 배포할 수 있는 기술적 민첩성(Agility)을 갖추어야** 합니다**.**



서비스 독립성, 우리 프로젝트에서 확인해보기

PART 1 :: CASE 02 · APPLY TO OUR PROJECT

--apply

Netflix처럼 Chaos Monkey까지는 못 만들어도, 우리 실습에서 서비스 독립성을 직접 실험해볼 수 있습니다.

1. docker-compose에서 enrollment-service 컨테이너만 잠깐 중지해보기
2. 그 상태에서 course-service의 과목 조회 API가 여전히 정상 응답하는지 확인하기
3. 정상이라면 서비스가 실제로 독립돼 있다는 뜻, 에러가 전파된다면 어디서 결합돼 있는지 찾아보기

이 실험 한 번이면 "MSA가 왜 독립적인가"를 코드로 체감할 수 있습니다.

Amazon Prime Video, 다시 모놀리식으로

PART 1 :: CASE 03/05

--case · msa is not silver bullet

Prime Video의 영상 품질 분석(VQA) 서비스는 여러 마이크로서비스와 AWS Step Functions로 오케스트레이션되어 있었는데, 프레임을 주고받기 위해 S3를 중간 저장소로 쓰는 구조가 비용과 확장성의 발목을 잡았습니다. 팀은 이 서비스를 하나의 애플리케이션으로 합치고 메모리 내에서 데이터를 주고받도록 재설계했습니다.

Prime Video 전체가 MSA를 버린 것은 아닙니다. 딱 이 서비스 하나에는 더 잘 맞는 구조가 모놀리식이었을 뿐입니다.

90%

인프라 비용 절감

오케스트레이션 방식 → 단일 프로세스 내 처리로 전환

왜 하필 이 서비스만 예외였나?

PART 1 :: CASE 03 · DEEP DIVE

① 아키텍처 선택 | ② 우리 프로젝트 | ③ 실수와 극복 | ④ 자가진단

--deep-dive

VQA 서비스는 초당 수많은 영상 프레임을 아주 짧은 단위 작업으로 처리해야 했습니다. 이런 "고빈도·초경량 작업"을 마이크로서비스로 나누면, 서비스 호출마다 발생하는 네트워크 통신과 오케스트레이션 상태 전이 비용이 실제 작업 비용보다 커지는 역전 현상이 생깁니다. 반대로 처리 단위가 크고 호출 빈도가 낮은 서비스라면 MSA의 이점이 비용보다 훨씬 큽니다.

일반적인 서비스

호출 빈도 낮음, 처리 단위 큼

MSA 이점 > 비용

VQA 같은 초경량·고빈도 작업

초당 수백 회 호출, 처리 단위 매우 작음

오케스트레이션 비용 > 작업 비용

언제 통합을 고려해야 하는가?

PART 1 :: CASE 03 · APPLY TO OUR PROJECT

--apply

우리 실습에서는 서비스 구조가 이미 주어져 있지만, 실무에서는 아래 신호가 보이면 통합을 검토합니다.

- 서비스 간 호출이 초당 수십~수백 회로 매우 잦다
- 호출 하나하나의 실제 작업량은 아주 작다 (수 ms 수준)
- 서비스 간 데이터 전달 비용(직렬화·네트워크)이 실제 연산 비용보다 크다
- 두 서비스가 항상 같이 배포되고 있어 독립 배포의 이점이 없다

네 가지 중 두 개 이상 해당한다면, "쪼개는 것이 항상 좋다"는 가정을 의심해볼 시점입니다.

Shopify, "위풍당당한 모놀리스"

PART 1 :: CASE 04/05

--case · modular monolith

Shopify는 세계 최대 규모의 Ruby on Rails 코드베이스 중 하나(280만 줄 이상)를 1,000명이 넘는 개발자가 함께 다룹니다. 이들은 이 거대한 코드를 마이크로서비스로 쪼개는 대신, 하나의 배포 단위(모놀리스)는 유지하면서 내부를 "컴포넌트"로 나누고 그 경계를 도구로 강제하는 길을 택했습니다. 이를 스스로 "Majestic Monolith(위풍당당한 모놀리스)"라 부릅니다.

280만+ 줄의 Rails 코드

1,000명+ 함께 작업하는 개발자

"쪼개지 않고, 경계만 명확히 한다"

경계를 "도구"로 강제하다

PART 1 :: CASE 04 · DEEP DIVE

--deep-dive

Shopify는 컴포넌트 경계를 문서나 팀 규칙으로만 정하지 않았습니다. 한 컴포넌트가 다른 컴포넌트의 내부 코드를 직접 참조하면 빌드·검사 단계에서 자동으로 걸러내는 도구를 자체적으로 만들어, "실수로 경계를 넘는 코드"가 애초에 병합되지 못하도록 했습니다. 사람이 코드 리뷰에서 매번 걸러내는 대신, 도구가 규칙을 강제하는 방식을 택한 것입니다.

"경계는 규칙이 아니라 도구로 지켜야 한다" → 이것이 Shopify 모듈러 모놀리식의 핵심입니다.

우리 프로젝트에서 경계 긋는 법

PART 1 :: CASE 04 · APPLY TO OUR PROJECT

--apply

우리 실습의 user-service, course-service, enrollment-service, payment-service 자체가 이미 "경계가 강제된" 예시입니다. 팀 프로젝트에서도 같은 원칙을 적용해보세요.

1. 도메인 매핑표에서 정한 각 서비스가 어떤 데이터를 "소유"하는지 한 줄로 적어보기
2. 다른 서비스의 DB를 직접 조회하지 않고, 반드시 API로만 데이터를 요청하는지 확인하기
3. 한 화면이 여러 서비스의 API를 직접 조합하는지, Gateway를 거치는지 점검하기

이 세 가지만 지켜도 여러분의 프로젝트는 이미 "경계가 있는" 구조입니다.

그래서, 언제 무엇을 선택하는가?

PART 1 :: CASE 05/05

① 아키텍처 선택 | ② 우리 프로젝트 | ③ 실수와 극복 | ④ 자가진단

--framework · 선택기준

기준	모놀리식	모듈러 모놀리식	MSA
팀 규모	작음 (1~수십명)	중간~대규모	대규모, 여러 팀
배포 빈도	낮음, 한 번에 전체	중간	높음, 서비스별 독립
운영 복잡도	낮음	낮음~중간	높음
초기 개발 속도	빠름	빠름~보통	느림 (초기 셋업 큼)

AI 서비스는 모델 교체와 요구사항 변화가 특히 잦다는 특성이 있어, MSA와 잘 맞는 경우가 많습니다. 이번 실습이 MSA로 설계된 이유이지, MSA가 모든 상황의 정답이라는 뜻은 아닙니다.

표로 안 보이면, 순서도로

PART 1 :: CASE 05 · DEEP DIVE

① 아키텍처 선택 | ② 우리 프로젝트 | ③ 실수와 극복 | ④ 자가진단

--deep-dive

Q1 팀이 5명 이하이고 기능도 단순한가?

예 → 모놀리식　아니오 → 다음 질문으로

Q2 배포는 팀 전체가 한 번에 해도 괜찮은가?

예 → 모듈러 모놀리식 검토　아니오 → 다음 질문으로

Q3 서비스별 독립 배포·확장이 반드시 필요한가?

예 → MSA　아니오 → 모듈러 모놀리식으로 충분

우리 팀 워크시트 : 5분만 채워보기

PART 1 :: CASE 05 · APPLY TO OUR PROJECT

① 아키텍처 선택 | ② 우리 프로젝트 | ③ 실수와 극복 | ④ 자가진단

--apply

1. 우리 팀 규모: _____명
2. 우리가 목표로 하는 배포 빈도: _____
3. 서비스 간 결합도가 가장 높을 것 같은 두 기능: _____ , _____
4. 우리가 MSA를 선택한 이유, 한 문장으로: _____

이 워크시트를 채우고 나면, 왜 우리 실습이 MSA로 설계됐는지 스스로 설명할 수 있게 됩니다.

우리 실습 프로젝트에서는 이렇게 작동한다

PART 2 :: CASE 01/01

--our-project

1

스프린트 피드백으로 방향을 바꾸다

스프린트1(회원·과목·수강신청)을 먼저 완성해 시연하고 나면, 그 피드백을 보고 스프린트2의 우선순위를 조정할 수 있습니다. 처음 계획을 끝까지 고집하지 않아도 되는 것이 Agile의 유연성입니다.

2

서비스가 나뉘어 있어 팀원이 기다리지 않는다

백엔드 담당이 결제 서비스를 만드는 동안, 프론트엔드 담당은 Swagger 문서만 보고 화면을 미리 만들 수 있습니다. 서비스가 독립된 MSA 구조이기 때문에 가능한 병렬 협업입니다.

스프린트1 시연, 실제로는 이렇게 흘러간다

PART 2 :: DEEP DIVE

① 아키텍처 선택 | ② 우리 프로젝트 | ③ 실수와 극복 | ④ 자가진단

--deep-dive

1주차

목표 확정

스프린트1 범위를 회원·과목·수강신청으로 확정

→

2~3주차

핵심 기능 구현

매일 짧게 진행 상황을 공유하며 개발

→

4주차

시연 & 피드백

실사용자(강사·학생 역할) 대상으로 시연, 의견 수집

→

반영

계획 재조정

"검색이 더 급하다" 의견 → 스프린트2에 검색 추가, 결제 고도화는 후순위

계획을 지키는 것보다, 계획을 언제 바꿀지 아는 것이 Agile의 실력입니다.

역할별 산출물표 : 스프린트마다 이렇게 제출

PART 2 :: APPLY TO OUR PROJECT

--apply

역할	스프린트1 산출물	스프린트2 산출물
기획	Pain Point 정의서, 도메인 매핑표	피드백 기반 우선순위 재조정안
백엔드	핵심 API 구현 + Swagger 문서	결제 · 이벤트 연동 API
프론트엔드	회원가입 · 조회 화면	결제 흐름 · 상태 반영 화면

표의 각 칸이 채워지지 않았다면, 그 역할은 이번 스프린트에서 할 일이 명확하지 않다는 신호입니다.

이번 과정, 이런 팀들이 있었습니다

PART 3 :: CASE 01/01

--lessons-learned

목업으로 도망간 팀

Swagger 연결법을 몰라 목업 데이터로만 화면을 완성 → Try it out으로 하나씩 확인하며 실제 연결에 성공한 팀과 대비

인프라까지 이해하려던 팀

Eureka·인증 코드를 한 줄씩 읽으려다 자괴감 → "알아야 할 것 vs 몰라도 되는 것" 기준을 알고 관점이 바뀜

역할 분담이 없었던 팀

한 사람이 기획·백엔드·프론트를 전부 떠안음 → API 명세를 공통 약속으로 삼아 역할을 나눈 팀과 대비

원격 브로드캐스팅이 어려운 친구

실시간 질문이 어려운 브로드캐스팅 반 → Slack Polly 피드백 기반 야간 보강으로 격차 해소

가장 흔한 두 실수, 자세히 보기

PART 3 :: DEEP DIVE

① 아키텍처 선택 | ② 우리 프로젝트 | ③ 실수와 극복 | ④ 자가진단

--deep-dive

목업으로 도망간 팀

Day 1 : Swagger 첫 화면을 보고 당황, "일단 화면 데이터는 하드코딩하고 넘어가자"
Day2 : 발표, 겉보기엔 동작하지만 새로 고침하면 다시 처음 데이터.

원인은 단 한 걸음, Try it out을 눌러보지 않은 것뿐이었습니다.

인프라까지 이해하려던 팀

Eureka 설정 파일을 3시간째 들여다보다 "나는 개발자 자격이 없나보다"라는 말이 나옴

야간 보강에서 "알아야 할 것 vs 몰라도 되는 것" 표를 보고 나서 "이걸 진작 알았으면..."

다음 날부터 API 명세만 보고 화면을 완성

우리 팀 예방 체크리스트

PART 3 :: APPLY TO OUR PROJECT

① 아키텍처 선택 | ② 우리 프로젝트 | ③ 실수와 극복 | ④ 자가진단

--apply

- [ ] 팀 전원이 Swagger UI에서 최소 1개 API를 Try it out 해봤는가
- [ ] "몰라도 되는 것" 표를 팀원 모두가 읽었는가
- [ ] 팀 안에서 API 명세 문서를 공유하고 있는가
- [ ] 원격 · 오프라인 상관없이 질문을 남길 수 있는 채널이 있는가

네 개 중 하나라도 아직이라면, 발표 전에 그 항목부터 채우세요.

우리 팀은 지금 어디에 있는가

- ✓ 우리 팀은 왜 MSA를 선택했는지 한 문장으로 설명할 수 있는가?
- ✓ Swagger UI로 실제 연결을 완료했는가, 아직 목업 상태인가?
- ✓ 스프린트1의 피드백을 스프린트2 계획에 반영했는가?
- ✓ 역할별 산출물이 API 명세로 서로 연결되는가?

더 알아보기 → 실습 가이드 문서와 "부록2. 자주 묻는 질문(Q&A)"을 함께 참고하세요.